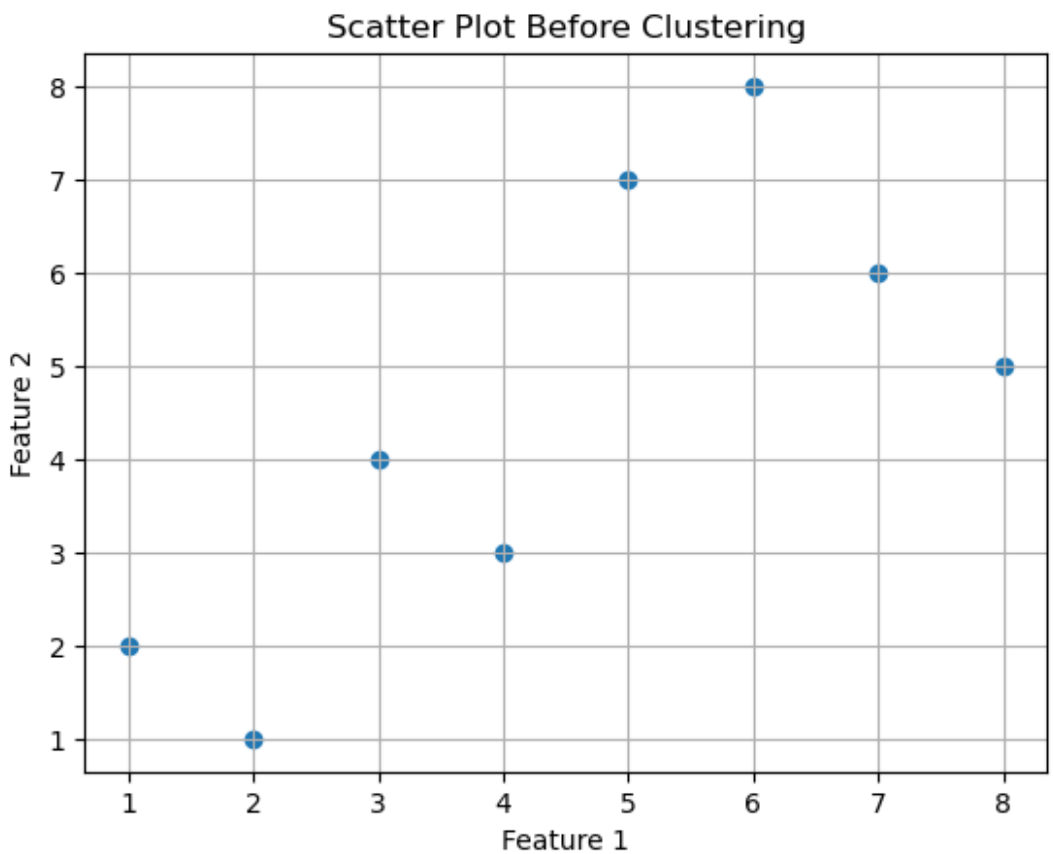
```python
# DATASET FOR CLUSTERING DEMONSTRATION
# NumPy is required here for numerical data handling
import numpy as np

X = np.array([
    [1, 2], [2, 1], [3, 4],
    [5, 7], [6, 8], [7, 6],
    [8, 5], [4, 3]
])
print(X)
```

```
[[1 2]
 [2 1]
 [3 4]
 [5 7]
 [6 8]
 [7 6]
 [8 5]
 [4 3]]
```

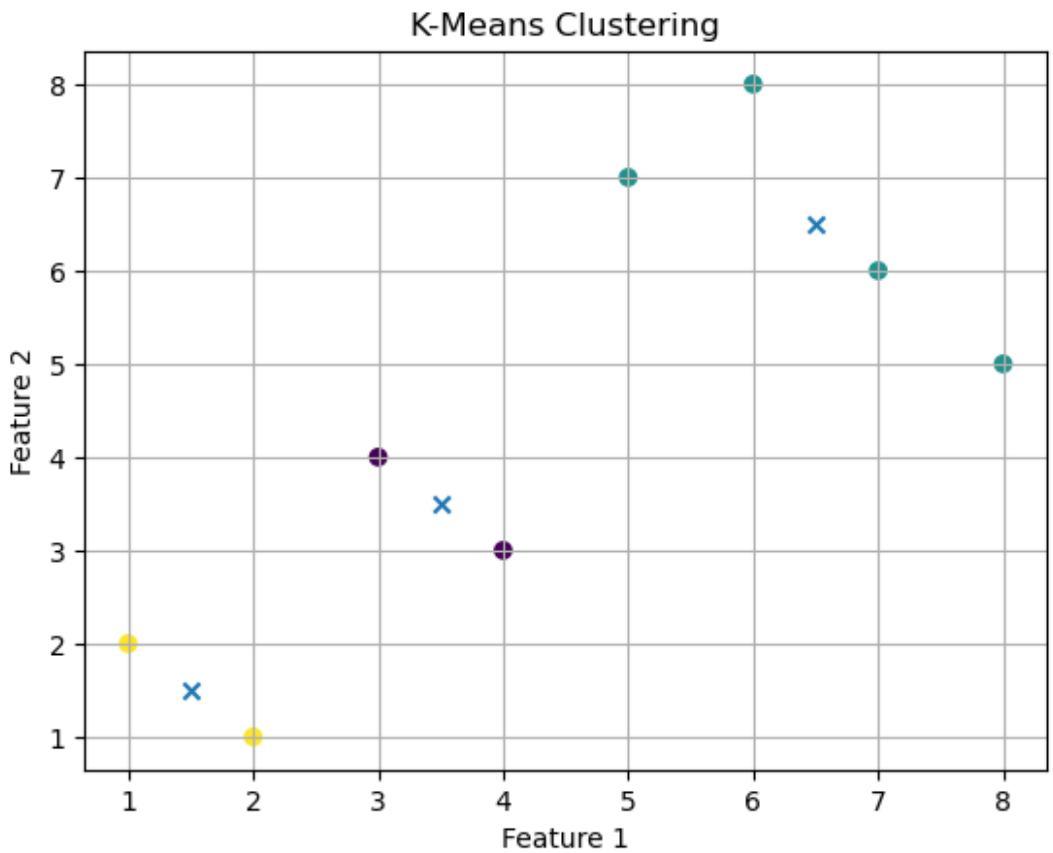
```python
# VISUALIZATION BEFORE CLUSTERING
# Matplotlib is needed only for plotting
import matplotlib.pyplot as plt

plt.scatter(X[:, 0], X[:, 1])
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Scatter Plot Before Clustering")
plt.grid(True)
plt.show()
```




```
# K-MEANS CLUSTERING
# Scikit-learn KMeans is required here
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=3, random_state=42)
labels = kmeans.fit_predict(X)

plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    marker='x'
)
plt.title("K-Means Clustering")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.grid(True)
plt.show()
```

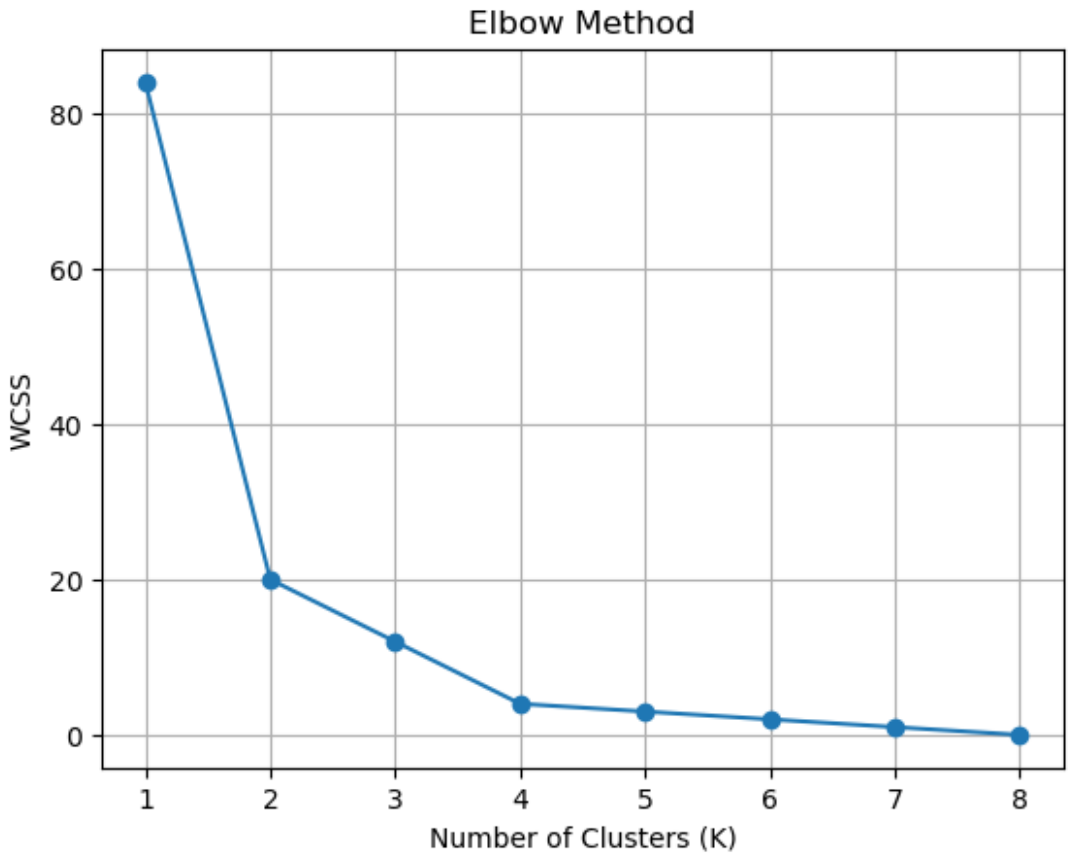
```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\
_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on
Windows with MKL, when there are less chunks than available threads.
You can avoid it by setting the environment variable
```

```
OMP_NUM_THREADS=1.
  warnings.warn(
```

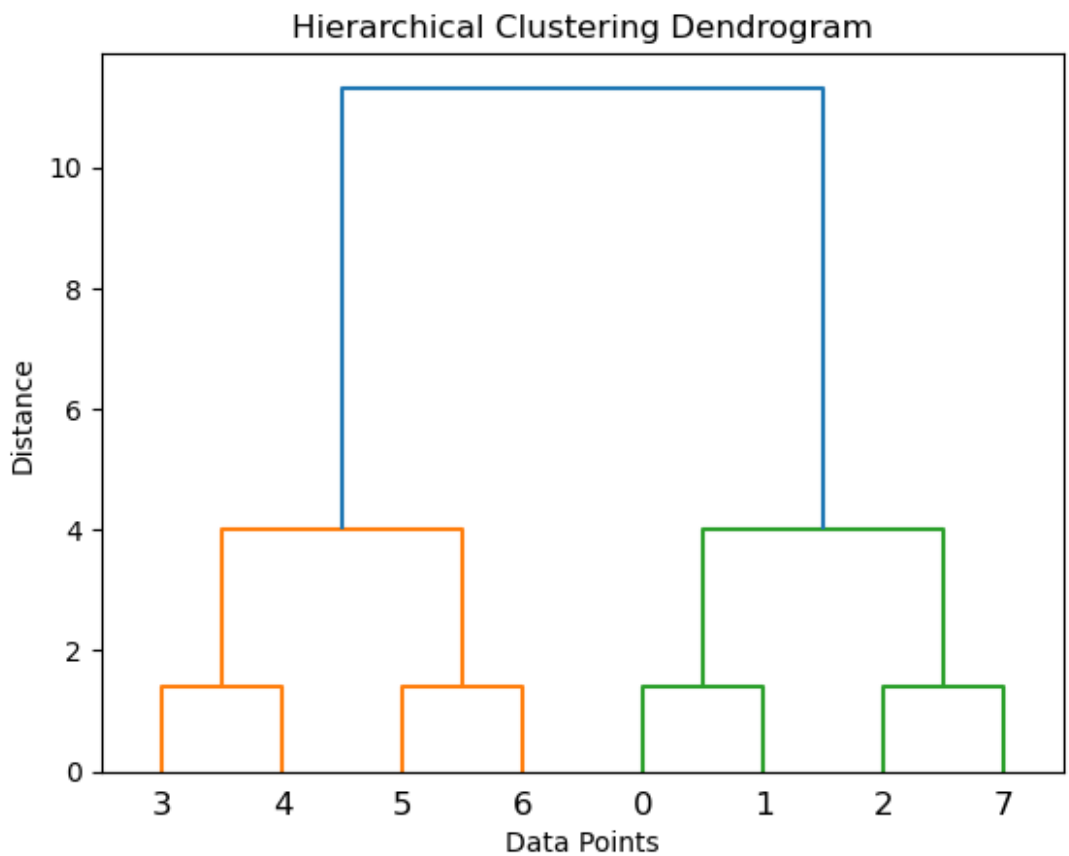



```
# ELBOW METHOD FOR OPTIMAL K
"""
Elbow Method:
- Plot K vs WCSS
- Elbow point gives optimal K
"""

wcss = []

for k in range(1, 9):
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X)
    wcss.append(km.inertia_)

plt.plot(range(1, 9), wcss, marker='o')
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.grid(True)
plt.show()
```

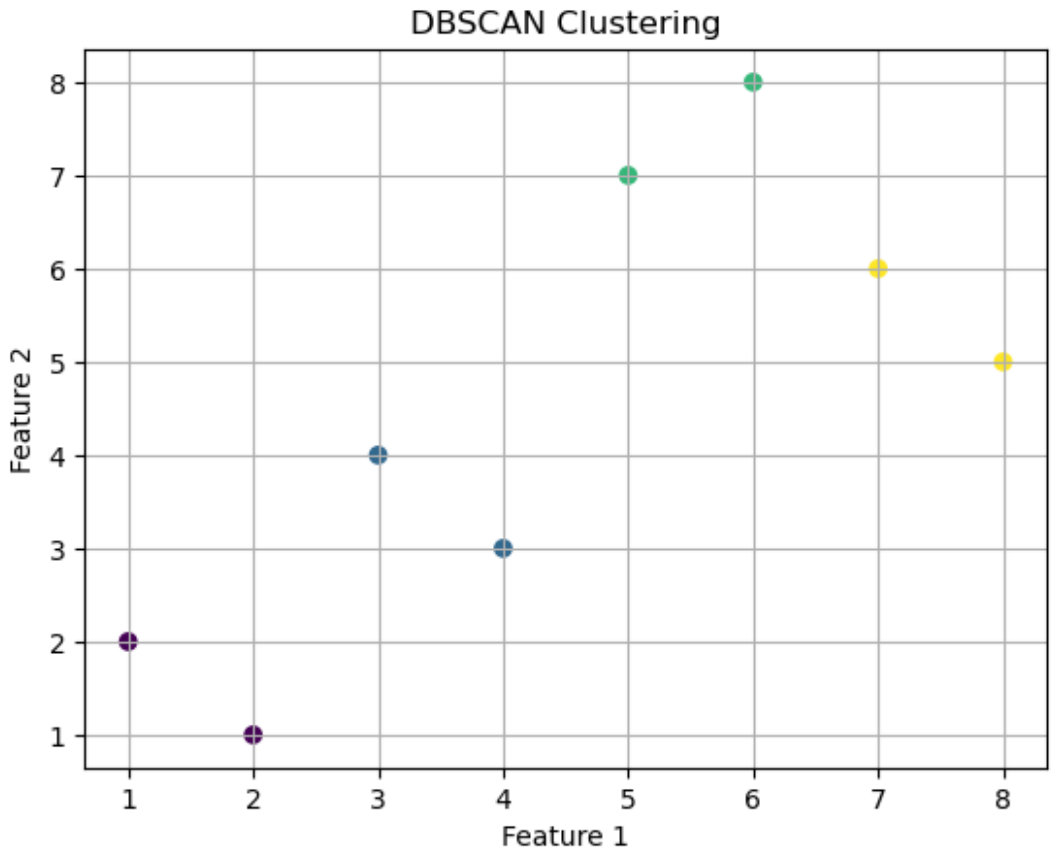
```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\
_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on
Windows with MKL, when there are less chunks than available threads.
You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.
  warnings.warn(
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\
_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on
Windows with MKL, when there are less chunks than available threads.
You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.
  warnings.warn(
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\
_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on
Windows with MKL, when there are less chunks than available threads.
You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.
  warnings.warn(
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\
_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on
Windows with MKL, when there are less chunks than available threads.
You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.
  warnings.warn(
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\
_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on
Windows with MKL, when there are less chunks than available threads.
You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.
  warnings.warn(
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\
_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on
Windows with MKL, when there are less chunks than available threads.
You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.
  warnings.warn(
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\
_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on
Windows with MKL, when there are less chunks than available threads.
You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.
  warnings.warn(
```




```
# Why this warning appears:
# - Scikit-learn uses Intel MKL
# - On Windows, KMeans may create too many threads
# - This causes a harmless memory-leak warning
# - Clustering results (WCSS and elbow plot) remain correct

# Best fix
# Limit the number of threads used by MKL

import os
os.environ["OMP_NUM_THREADS"] = "1"

# HIERARCHICAL CLUSTERING
# SciPy is required here for hierarchical clustering
from scipy.cluster.hierarchy import dendrogram, linkage

Z = linkage(X, method='ward')

dendrogram(Z)
plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()
```




```
# DENSITY-BASED CLUSTERING (DBSCAN)
# DBSCAN imported only when needed
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=1.5, min_samples=2)
db_labels = dbscan.fit_predict(X)

plt.scatter(X[:, 0], X[:, 1], c=db_labels)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("DBSCAN Clustering")
plt.grid(True)
plt.show()
```




```python
# ASSOCIATION RULE LEARNING
"""
Market Basket Analysis
- Discover frequent itemsets
- Generate association rules
"""

transactions = [
    {"Bread", "Milk", "Egg", "Butter", "Salt", "Apple"},
    {"Bread", "Milk", "Egg", "Apple"},
    {"Bread", "Milk", "Butter", "Apple"},
    {"Milk", "Egg", "Butter", "Apple"},
    {"Bread", "Egg", "Salt"},
    {"Bread", "Milk", "Egg", "Apple"}
]
print(transactions)
```

```
[{'Milk', 'Bread', 'Butter', 'Egg', 'Apple', 'Salt'}, {'Milk',
'Bread', 'Apple', 'Egg'}, {'Milk', 'Bread', 'Apple', 'Butter'},
{'Milk', 'Butter', 'Apple', 'Egg'}, {'Egg', 'Bread', 'Salt'}, {'Milk',
'Bread', 'Apple', 'Egg'}]
```

```python
# To print transactions in dataframe
# Pandas is required only at this stage
import pandas as pd
# Convert transactions into a DataFrame (one-hot encoding style)
items = sorted(set().union(*transactions))

data = []
for t in transactions:
    row = {item: (1 if item in t else 0) for item in items}
    data.append(row)

df = pd.DataFrame(data)
df.index = range(1, len(df) + 1)
df.index.name = "Transaction"

print(df)
```

```
             Apple  Bread  Butter  Egg  Milk  Salt
Transaction
1                1      1       1    1     1     1
2                1      1       0    1     1     0
3                1      1       1    0     1     0
4                1      0       1    1     1     0
5                0      1       0    1     0     1
6                1      1       0    1     1     0
```

```python
# SUPPORT AND CONFIDENCE FUNCTIONS
def support(itemset, transactions):
    return sum(1 for t in transactions if itemset.issubset(t)) /
len(transactions)

def confidence(X, Y, transactions):
    return support(X.union(Y), transactions) / support(X,
transactions)

# Example rules
print("Support(Bread):", support({"Bread"}, transactions))
print("Confidence(Bread → Milk):",
      confidence({"Bread"}, {"Milk"}, transactions))
print("Confidence(Bread, Milk → Egg):",
      confidence({"Bread", "Milk"}, {"Egg"}, transactions))
```

```
Support(Bread): 0.8333333333333334
Confidence(Bread → Milk): 0.7999999999999999
Confidence(Bread, Milk → Egg): 0.75
```

```python
# APRIORI PRINCIPLE
"""
Apriori Principle:
- If an itemset is frequent, all its subsets must be frequent
- If an itemset is infrequent, all its supersets are infrequent
```

```
Steps:
1. Generate frequent itemsets using minimum support
2. Generate strong rules using minimum confidence
"""

'\nApriori Principle:\n- If an itemset is frequent, all its subsets
must be frequent\n- If an itemset is infrequent, all its supersets are
infrequent\n\nSteps:\n1. Generate frequent itemsets using minimum
support\n2. Generate strong rules using minimum confidence\n'

# Minimum support threshold
min_support = 0.5

# ----------------------------------
# Support function
# ----------------------------------
def support(itemset):
    return sum(1 for t in transactions if itemset.issubset(t)) /
len(transactions)

# ----------------------------------
# Apriori Principle demonstration
# ----------------------------------
# Step 1: Find frequent 1-itemsets
items = sorted(set().union(*transactions))
freq_1 = [ {i} for i in items if support({i}) >= min_support ]

print("Frequent 1-itemsets:")
for i in freq_1:
    print(i, "Support =", support(i))

# Step 2: Generate candidate 2-itemsets
from itertools import combinations

candidates_2 = [ set(c) for c in combinations([list(i)[0] for i in
freq_1], 2) ]

# Apply Apriori pruning
freq_2 = [ c for c in candidates_2 if support(c) >= min_support ]

print("\nFrequent 2-itemsets (after Apriori pruning):")
for c in freq_2:
    print(c, "Support =", support(c))
```

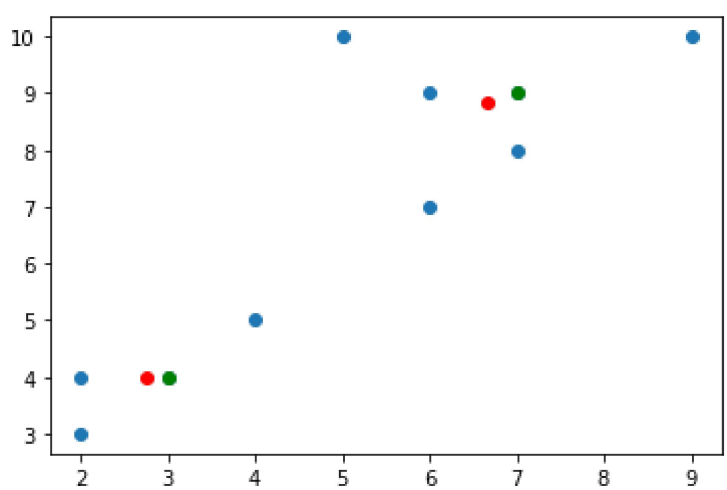
```
Frequent 1-itemsets:
{'Apple'} Support = 0.8333333333333334
{'Bread'} Support = 0.8333333333333334
{'Butter'} Support = 0.5
{'Egg'} Support = 0.8333333333333334
{'Milk'} Support = 0.8333333333333334
```

```
Frequent 2-itemsets (after Apriori pruning):
{'Bread', 'Apple'} Support = 0.6666666666666666
{'Butter', 'Apple'} Support = 0.5
{'Egg', 'Apple'} Support = 0.6666666666666666
{'Milk', 'Apple'} Support = 0.8333333333333334
{'Egg', 'Bread'} Support = 0.6666666666666666
{'Milk', 'Bread'} Support = 0.6666666666666666
{'Milk', 'Butter'} Support = 0.5
{'Egg', 'Milk'} Support = 0.6666666666666666
```

In [1]:
```
pip install scikit-learn-extra
```

```
Collecting scikit-learn-extra
  Downloading scikit_learn_extra-0.3.0-cp39-cp39-win_amd64.whl (346 kB)Note: you may nee
d to restart the kernel to use updated packages.
Requirement already satisfied: scikit-learn>=0.23.0 in c:\users\iter\anaconda3\lib\site-
packages (from scikit-learn-extra) (0.24.2)
Requirement already satisfied: numpy>=1.13.3 in c:\users\iter\anaconda3\lib\site-package
s (from scikit-learn-extra) (1.20.3)
Requirement already satisfied: scipy>=0.19.1 in c:\users\iter\anaconda3\lib\site-package
s (from scikit-learn-extra) (1.7.1)
Requirement already satisfied: joblib>=0.11 in c:\users\iter\anaconda3\lib\site-packages
(from scikit-learn>=0.23.0->scikit-learn-extra) (1.1.0)
Requirement already satisfied: threadpoolctl>=2.0.0 in c:\users\iter\anaconda3\lib\site-
packages (from scikit-learn>=0.23.0->scikit-learn-extra) (2.2.0)
Installing collected packages: scikit-learn-extra
Successfully installed scikit-learn-extra-0.3.0
```

In [2]:
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn_extra.cluster import KMedoids
data = [[2,4],[2,3],[3,4],[4,5],[6,7],[7,8],[9,10],[5,10],[6,9],[7,9]]
df = pd.DataFrame(data, columns=['x','y'])
# KMeans
kmean = KMeans(n_clusters=2, random_state=42)
kmean.fit(df)
centroids = kmean.cluster_centers_
# KMedoids
kmedoid = KMedoids(n_clusters=2, random_state=42)
kmedoid.fit(df)
medoids = kmedoid.cluster_centers_
plt.scatter(df['x'], df['y'], label='Data')
plt.scatter(centroids[:,0], centroids[:,1], color='red')
plt.scatter(medoids[:,0], medoids[:,1], color='green')
plt.show()
print("Centroids:", centroids)
print("Medoids:", medoids)
```




```
Centroids: [[6.66666667 8.83333333]
 [2.75       4.        ]]
```

```
Medoids: [[3. 4.]
 [7. 9.]]
```

In [ ]:

Ch10_Lab(part-1)

February 17, 2026

1 Chapter 10-Part 1

1. A very small example showing inputs → weights → sum → activation → output.

[1]:
```
import numpy as np

# inputs
x = np.array([0.5, 0.2, 0.1])

# weights
w = np.array([0.4, -0.7, 0.2])

# weighted sum
y_sum = np.dot(x, w)

# simple activation (identity)
output = y_sum

print("Weighted Sum:", y_sum)
print("Output:", output)
```

```
Weighted Sum: 0.08000000000000004
Output: 0.08000000000000004
```

2. Biological Neuron (Conceptual Simulation), mimicking dendrites → soma → axon idea.

[2]:
```
def biological_neuron(signals, threshold=1.0):
    soma = sum(signals)   # accumulation
    if soma >= threshold:
        return 1   # neuron fires
    return 0

signals = [0.3, 0.4, 0.5]
print("Neuron Output:", biological_neuron(signals))
```

```
Neuron Output: 1
```

3. Artificial neuron with weights + activation

[3]:
```python
import numpy as np

def artificial_neuron(inputs, weights, bias=0):
    y_sum = np.dot(inputs, weights) + bias
    return y_sum

x = np.array([1, 0, 1])
w = np.array([0.5, -0.6, 0.2])

print("Summation Output:", artificial_neuron(x, w, bias=0.1))
```

```
Summation Output: 0.7999999999999999
```

4. Activation Functions

[4]:
```python
import math

# #Identity function
def identity(x):
    return x

#Step / Threshold function
def step(x):
    return 1 if x >= 0 else 0

#ReLU function
def threshold(x, theta=0.5):
    return 1 if x >= theta else 0

#ReLU function
def relu(x):
    return max(0, x)

#Sigmoid function
def sigmoid(x, k=1):
    return 1 / (1 + math.exp(-k * x))

#Bipolar Sigmoid
def bipolar_sigmoid(x):
    return (2 / (1 + math.exp(-x))) - 1

#Hyperbolic Tangent
def tanh_n(x):
    return math.tanh(x)

# -------- Test Values --------
values = [-2, -1, 0, 0.5, 1, 2]
```

```python
print("Input Values:", values)
print("\n--- Identity Function ---")
for v in values:
    print(v, "->", identity(v))

print("\n--- Step Function ---")
for v in values:
    print(v, "->", step(v))

print("\n--- Threshold Function (theta=0.5) ---")
for v in values:
    print(v, "->", threshold(v))

print("\n--- ReLU Function ---")
for v in values:
    print(v, "->", relu(v))

print("\n--- Sigmoid Function ---")
for v in values:
    print(v, "->", round(sigmoid(v), 4))

print("\n--- Bipolar Sigmoid Function ---")
for v in values:
    print(v, "->", round(bipolar_sigmoid(v), 4))

print("\n--- Tanh Function ---")
for v in values:
    print(v, "->", round(tanh_n(v), 4))
```

```
Input Values: [-2, -1, 0, 0.5, 1, 2]

--- Identity Function ---
-2 -> -2
-1 -> -1
0 -> 0
0.5 -> 0.5
1 -> 1
2 -> 2

--- Step Function ---
-2 -> 0
-1 -> 0
0 -> 1
0.5 -> 1
1 -> 1
2 -> 1
```

```
--- Threshold Function (theta=0.5) ---
-2 -> 0
-1 -> 0
0 -> 0
0.5 -> 1
1 -> 1
2 -> 1

--- ReLU Function ---
-2 -> 0
-1 -> 0
0 -> 0
0.5 -> 0.5
1 -> 1
2 -> 2

--- Sigmoid Function ---
-2 -> 0.1192
-1 -> 0.2689
0 -> 0.5
0.5 -> 0.6225
1 -> 0.7311
2 -> 0.8808

--- Bipolar Sigmoid Function ---
-2 -> -0.7616
-1 -> -0.4621
0 -> 0.0
0.5 -> 0.2449
1 -> 0.4621
2 -> 0.7616

--- Tanh Function ---
-2 -> -0.964
-1 -> -0.7616
0 -> 0.0
0.5 -> 0.4621
1 -> 0.7616
2 -> 0.964
```

5. McCulloch–Pitts Neuron (OR Logic Example)

[5]:
```python
def mcp_neuron(x1, x2, w1=1, w2=1, threshold=1):
    y_sum = x1*w1 + x2*w2
    return 1 if y_sum >= threshold else 0

# OR gate truth table
for x1 in [0,1]:
```

```
    for x2 in [0,1]:
        print(x1, x2, "->", mcp_neuron(x1, x2))
```

```
0 0 -> 0
0 1 -> 1
1 0 -> 1
1 1 -> 1
```

6. Rosenblatt Perceptron (Binary Classifier)

[6]:
```
import numpy as np

class Perceptron:
    def __init__(self, lr=0.1, epochs=10):
        self.lr = lr
        self.epochs = epochs

    def activation(self, x):
        return 1 if x >= 0 else -1

    def fit(self, X, y):
        self.weights = np.zeros(X.shape[1])
        self.bias = 0

        for _ in range(self.epochs):
            for i in range(len(X)):
                linear = np.dot(X[i], self.weights) + self.bias
                y_pred = self.activation(linear)

                update = self.lr * (y[i] - y_pred)
                self.weights += update * X[i]
                self.bias += update

    def predict(self, X):
        linear = np.dot(X, self.weights) + self.bias
        return self.activation(linear)


# Sample data
X = np.array([[2,3],[1,1],[2,1],[-1,-2]])
y = np.array([1,1,-1,-1])

p = Perceptron()
p.fit(X, y)
print("Prediction:", p.predict(np.array([2,2])))
```

```
Prediction: 1
```

Explanation: 1. Perceptron- A Perceptron is the simplest neural network model used for binary

classification (two classes only). One neuron that learns to separate two groups with a straight line. 2. 'self' in Python- 'self' is not neural network theory, it is Python syntax used inside a class. It refers to the current object (instance) of the class. Think of it as "this object". Example:

[8]:
```
class Student:
    def __init__(self, name):
        self.name = name
```

Here: 'self.name' stores the name inside that object. Without 'self', Python won't know where to store it. In Perceptron code:'self.weights, self.bias', means weights and bias belong to that perceptron object.

3. Epochs:= One full training cycle over the entire dataset. If you have 100 data points: 1 epoch → model sees all 100 once, 10 epochs → model sees all 100 ten times. Why Multiple Epochs?- Because learning improves gradually. More epochs → better adjustment of weights (up to a limit). Simple Analogy- Studying for exams: 1 reading = 1 epoch, 5 revisions = 5 epochs

7. ADALINE (Delta Rule / LMS)

[7]:
```
import numpy as np

class Adaline:
    def __init__(self, lr=0.01, epochs=20):
        self.lr = lr
        self.epochs = epochs

    def fit(self, X, y):
        self.weights = np.zeros(X.shape[1])
        self.bias = 0

        for _ in range(self.epochs):
            for i in range(len(X)):
                y_sum = np.dot(X[i], self.weights) + self.bias
                error = y[i] - y_sum

                self.weights += self.lr * error * X[i]
                self.bias += self.lr * error

    def predict(self, X):
        y_sum = np.dot(X, self.weights) + self.bias
        return 1 if y_sum >= 0 else -1


X = np.array([[1,2],[2,3],[3,4],[-1,-2]])
y = np.array([1,1,1,-1])

a = Adaline()
a.fit(X, y)
print("Prediction:", a.predict(np.array([2,2])))
```

```
Prediction: 1
```

[ ]:

```

```

Ch10_Lab (part-2)

February 19, 2026

Single Layer Feed Forward Network (Perceptron) Aim: Implement a single-layer neural network for binary classification.

[1]:
```
import numpy as np
from sklearn.linear_model import Perceptron
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification
from sklearn.metrics import accuracy_score

# Generate dataset
X, y = make_classification(n_samples=200, n_features=2,
                           n_redundant=0, n_clusters_per_class=1)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Create model
model = Perceptron(max_iter=1000, eta0=0.1)

# Train
model.fit(X_train, y_train)

# Test
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.7666666666666667
```

Multi-Layer Feed Forward Network (Backpropagation) Aim: Implement Multi-Layer Perceptron using backpropagation.

[8]:
```
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification
from sklearn.metrics import accuracy_score

# Step 1: Generate dataset
```

```
X, y = make_classification(n_samples=300,
                           n_features=4,
                           n_informative=4,
                           n_redundant=0,
                           random_state=1)

# Step 2: Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Step 3: Create Multi-Layer Feed Forward Network
model = MLPClassifier(hidden_layer_sizes=(10, 5),   # two hidden layers
                      activation='relu',
                      learning_rate_init=0.01,
                      max_iter=500)

# Step 4: Train the model
model.fit(X_train, y_train)

# Step 5: Predict
y_pred = model.predict(X_test)

# Step 6: Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.9111111111111111
```

LAB 3: Backpropagation (Manual Implementation) Aim: Implement basic backpropagation using NumPy (XOR Problem).

[3]:
```
import numpy as np

# XOR dataset
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]])

# Initialize weights
np.random.seed(1)
weights = np.random.rand(2,1)
learning_rate = 0.1

# Sigmoid function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# Training
```

```
for epoch in range(5000):
    # Forward
    output = sigmoid(np.dot(X, weights))

    # Error
    error = y - output

    # Backward
    adjustments = learning_rate * np.dot(X.T, error *␣
 ↪sigmoid_derivative(output))
    weights += adjustments

print("Final Output:")
print(output)
```

```
Final Output:
[[0.5]
 [0.5]
 [0.5]
 [0.5]]
```

Deep Learning Model (3+ Hidden Layers) Aim: Implement Deep Neural Network.

[7]:
```
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification
from sklearn.metrics import accuracy_score

# Generate dataset
X, y = make_classification(n_samples=300,
                           n_features=4,
                           n_informative=4,
                           n_redundant=0,
                           random_state=1)

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Deep Neural Network (3 hidden layers)
model = MLPClassifier(hidden_layer_sizes=(64, 32, 16),
                      activation='relu',
                      learning_rate_init=0.01,
                      max_iter=500)

# Train
model.fit(X_train, y_train)

# Predict
```

```python
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 1.0
```

[ ]:

```python

```

In [2]:
```python
###Implementation of ICA
###This shows how ICA separates mixed signals.

import numpy as np
from sklearn.decomposition import FastICA

# Step 1: Create 2 simple independent signals
S = np.array([[1, 1],
              [2, -1],
              [3, 1],
              [4, -1]])

# Step 2: Mix them
A = np.array([[1, 2],
              [3, 4]])
X = S @ A.T   # Observed mixed data

# Step 3: Apply ICA
ica = FastICA(n_components=2, random_state=0)
U = ica.fit_transform(X)

print("Original Sources:\n", S)
print("Recovered Sources (approx):\n", U)
```

```
Original Sources:
 [[ 1  1]
 [ 2 -1]
 [ 3  1]
 [ 4 -1]]
Recovered Sources (approx):
 [[-1.  1.]
 [-1. -1.]
 [ 1.  1.]
 [ 1. -1.]]
```

In [12]:
```python
### Implementation of Autoencoder
###This code shows how an autoencoder compresses 4 input features
###into 2 hidden features and then reconstructs the original data.

import numpy as np
from sklearn.neural_network import MLPRegressor

# Step 1: Create simple data (4 features)
X = np.array([[1, 2, 3, 4],
              [2, 3, 4, 5],
              [3, 4, 5, 6],
              [4, 5, 6, 7]], dtype=float)

# Step 2: Create Autoencoder (4 -> 2 -> 4)
autoencoder = MLPRegressor(hidden_layer_sizes=(2,),
                           activation="relu",
                           max_iter=5000,
                           random_state=0)
```

```
# Step 3: Train to reproduce input
autoencoder.fit(X, X)

# Step 4: Reconstruct
X_reconstructed = autoencoder.predict(X)

# Step 5: Print 2D hidden representation (the "4 -> 2" part)
W = autoencoder.coefs_[0]       # (4 x 2) input->hidden weights
b = autoencoder.intercepts_[0]  # (2,) hidden biases
hidden_2d = np.maximum(0, X @ W + b)  # ReLU: max(0, ...)

print("Original (4 features):\n", X)
print("\nHidden (2 features)  [this is the 4 -> 2 compression]:\n", np.round(h
print("\nReconstructed (4 features):\n", np.round(X_reconstructed, 2))
```

```
Original (4 features):
 [[1. 2. 3. 4.]
 [2. 3. 4. 5.]
 [3. 4. 5. 6.]
 [4. 5. 6. 7.]]

Hidden (2 features)  [this is the 4 -> 2 compression]:
 [[4.35 1.56]
 [5.81 2.11]
 [7.27 2.66]
 [8.74 3.21]]

Reconstructed (4 features):
 [[2.05 2.76 3.4  3.99]
 [2.52 3.33 4.09 5.  ]
 [2.99 3.89 4.78 6.  ]
 [3.45 4.45 5.47 7.  ]]
```

In [11]:

```
###Implementation of Active learning
###It shows how Active Learning improves a model by repeatedly selecting
###the most uncertain data points for labeling.

import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression

# Create simple dataset (fix parameters properly)
X, y = make_classification(n_samples=100,
                           n_features=2,
                           n_redundant=0,
                           n_informative=2,
                           random_state=0)

# Start with 10 labeled samples
labeled = np.arange(10)
unlabeled = np.arange(10, 100)

model = LogisticRegression(solver="liblinear")
```

```python
for i in range(5):   # 5 active queries
    model.fit(X[labeled], y[labeled])

    probs = model.predict_proba(X[unlabeled])[:, 1]

    # pick most uncertain sample
    query = np.argmin(np.abs(probs - 0.5))

    # move that sample to labeled set
    labeled = np.append(labeled, unlabeled[query])
    unlabeled = np.delete(unlabeled, query)

    print("Iteration:", i+1, "| Labeled size:", len(labeled))
```

```
Iteration: 1 | Labeled size: 11
Iteration: 2 | Labeled size: 12
Iteration: 3 | Labeled size: 13
Iteration: 4 | Labeled size: 14
Iteration: 5 | Labeled size: 15
```

In [5]:

```python
###Implementation of RBF
###This code shows how an RBF network uses Gaussian distance from fixed center
###to classify data by computing weighted combinations of radial basis functic


import numpy as np

# Simple 1D input data
X = np.array([[1], [2], [3], [4]])
y = np.array([0, 0, 1, 1])

# Choose 2 fixed RBF centers manually
centers = np.array([[1], [4]])
sigma = 1.0

# Gaussian RBF function
def rbf(x, c):
    return np.exp(-np.linalg.norm(x - c)**2 / (2 * sigma**2))

# Build hidden layer output
H = np.array([[rbf(x, c) for c in centers] for x in X])

# Solve for weights using simple linear equation
w = np.linalg.pinv(H) @ y   # Moore–Penrose inverse

# Predict
y_pred = H @ w
y_pred_class = (y_pred > 0.5).astype(int)

print("Predicted:", y_pred_class)
```

```
Predicted: [0 0 1 1]
```

In [6]:

```python
###Implementation of ECLAT algorithm
```

```python
###This code shows how the ECLAT algorithm finds frequent itemsets
###by intersecting transaction ID (TID) lists to compute support.


# Simple transactions
transactions = [
    {"A", "B"},
    {"A", "C"},
    {"B", "C"},
    {"A", "B"},
]

min_sup = 2

# Step 1: Build TID lists (vertical format)
tid = {}
for i, t in enumerate(transactions):
    for item in t:
        if item not in tid:
            tid[item] = set()
        tid[item].add(i)

print("TID lists:", tid)

# Step 2: Intersect manually (example: A and B)
AB = tid["A"] & tid["B"]   # core idea of ECLAT
print("Support of (A,B):", len(AB))

# Step 3: Check if frequent
if len(AB) >= min_sup:
    print("(A,B) is frequent")
```

```
TID lists: {'A': {0, 1, 3}, 'B': {0, 2, 3}, 'C': {1, 2}}
Support of (A,B): 2
(A,B) is frequent
```

In [7]:

```python
###Implementation of bagging

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier
from sklearn.metrics import accuracy_score

X, y = make_classification(n_samples=500, n_features=5, random_state=0)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0)

bag = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=20,      # number of bootstrap models
    bootstrap=True,
    random_state=0
)
bag.fit(Xtr, ytr)
```

```
print("Bagging accuracy:", accuracy_score(yte, bag.predict(Xte)))
```

```
Bagging accuracy: 0.9133333333333333
```

In [8]:

```
###Implementation of boosting using Adaboost


from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),  # weak learner (stump)
    n_estimators=50,
    learning_rate=1.0,
    random_state=0
)
ada.fit(Xtr, ytr)
print("AdaBoost accuracy:", accuracy_score(yte, ada.predict(Xte)))
```

```
AdaBoost accuracy: 0.8533333333333334
```

In [9]:

```
###Implementation of Gradient boosting classisfier

from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score

gbm = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, random_s
gbm.fit(Xtr, ytr)
print("GBM accuracy:", accuracy_score(yte, gbm.predict(Xte)))
```

```
GBM accuracy: 0.9266666666666666
```

In [10]:

```
###Implementation of Elastic net
###This code shows how Elastic Net regression combines L1 and L2 regularizatic
###to predict continuous values while controlling overfitting.


from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.linear_model import ElasticNet
from sklearn.metrics import mean_squared_error

# Create simple regression data
X, y = make_regression(n_samples=200, n_features=5, noise=10, random_state=0)

# Split
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0)

# Elastic Net model
model = ElasticNet(alpha=1.0, l1_ratio=0.5)
# alpha = regularization strength
# l1_ratio = balance (0=L2 only, 1=L1 only)

model.fit(Xtr, ytr)
```

```python
# Predict
y_pred = model.predict(Xte)

print("MSE:", mean_squared_error(yte, y_pred))
```

```
MSE: 3027.562035885066
```

In [ ]:

In [4]:
```
!pip install gymnasium
```

```
Requirement already satisfied: gymnasium in c:\users\hp\anaconda3\lib\site-pack
ages (1.2.3)
Requirement already satisfied: numpy>=1.21.0 in c:\users\hp\anaconda3\lib\site-
packages (from gymnasium) (2.1.3)
Requirement already satisfied: cloudpickle>=1.2.0 in c:\users\hp\anaconda3\lib\
site-packages (from gymnasium) (3.0.0)
Requirement already satisfied: typing-extensions>=4.3.0 in c:\users\hp\anaconda
3\lib\site-packages (from gymnasium) (4.12.2)
Requirement already satisfied: farama-notifications>=0.0.1 in c:\users\hp\anaco
nda3\lib\site-packages (from gymnasium) (0.0.4)
```

In [5]:
```
#Basic Reinforcement Learning Structure (Agent–Environment)
import numpy as np
import gymnasium as gym

# ============================================================
# 1. BASIC AGENT (Q-learning style)
# ============================================================
class SimpleAgent:
    def __init__(self, n_states, n_actions):
        self.q_table = np.zeros((n_states, n_actions))
        self.alpha = 0.1
        self.gamma = 0.9
        self.epsilon = 0.1

    def choose_action(self, state):
        if np.random.rand() < self.epsilon:
            return np.random.choice(len(self.q_table[state]))
        else:
            return np.argmax(self.q_table[state])

    def update(self, state, action, reward, next_state):
        best_next = np.max(self.q_table[next_state])
        self.q_table[state, action] += self.alpha * (
            reward + self.gamma * best_next - self.q_table[state, action]
        )

print("\n===== BASIC AGENT TEST =====")
agent = SimpleAgent(3, 2)
agent.update(0, 1, 10, 2)
print("Q-table:\n", agent.q_table)
```

```
===== BASIC AGENT TEST =====
Q-table:
 [[0. 1.]
 [0. 0.]
 [0. 0.]]
```

In [6]:
```
# Markov Decision Process (MDP Representation)
states = [0, 1, 2]
actions = [0, 1]
```

```
P = {
    0: {0: [(1, 1.0)], 1: [(2, 1.0)]},
    1: {0: [(0, 1.0)], 1: [(2, 1.0)]},
    2: {0: [(2, 1.0)], 1: [(2, 1.0)]}
}

R = {
    0: {0: 5, 1: 10},
    1: {0: -1, 1: 2},
    2: {0: 0, 1: 0}
}

print("\n===== MDP STRUCTURE =====")
print("States:", states)
print("Actions:", actions)
print("Transitions:", P)
print("Rewards:", R)
```

```
===== MDP STRUCTURE =====
States: [0, 1, 2]
Actions: [0, 1]
Transitions: {0: {0: [(1, 1.0)], 1: [(2, 1.0)]}, 1: {0: [(0, 1.0)], 1: [(2,
1.0)]}, 2: {0: [(2, 1.0)], 1: [(2, 1.0)]}}
Rewards: {0: {0: 5, 1: 10}, 1: {0: -1, 1: 2}, 2: {0: 0, 1: 0}}
```

In [7]:

```
#Policy Evaluation (Iterative)
def policy_evaluation(policy, P, R, gamma=0.9, theta=1e-5):
    V = np.zeros(len(policy))
    while True:
        delta = 0
        for s in range(len(policy)):
            v = V[s]
            a = policy[s]
            V[s] = R[s][a] + gamma * sum(
                prob * V[next_state] for next_state, prob in P[s][a]
            )
            delta = max(delta, abs(v - V[s]))
        if delta < theta:
            break
    return V

policy = [0, 1, 0]
V = policy_evaluation(policy, P, R)

print("\n===== POLICY EVALUATION =====")
for i, v in enumerate(V):
    print(f"State {i}: {v:.4f}")
```

```
===== POLICY EVALUATION =====
State 0: 6.8000
State 1: 2.0000
State 2: 0.0000
```

In [8]:

```
#Policy Improvement
```

```python
def policy_improvement(V, P, R, gamma=0.9):
    policy = np.zeros(len(V), dtype=int)
    for s in range(len(V)):
        action_values = []
        for a in P[s]:
            val = R[s][a] + gamma * sum(
                prob * V[next_state] for next_state, prob in P[s][a]
            )
            action_values.append(val)
        policy[s] = np.argmax(action_values)
    return policy

improved_policy = policy_improvement(V, P, R)

print("\n===== POLICY IMPROVEMENT =====")
print("Improved Policy:", improved_policy)
```

```
===== POLICY IMPROVEMENT =====
Improved Policy: [1 0 0]
```

In [9]:

```python
#Policy Iteration
def policy_iteration(P, R, gamma=0.9):
    policy = np.zeros(len(P), dtype=int)
    while True:
        V = policy_evaluation(policy, P, R, gamma)
        new_policy = policy_improvement(V, P, R, gamma)
        if np.array_equal(policy, new_policy):
            break
        policy = new_policy
    return policy, V

opt_policy, opt_V = policy_iteration(P, R)

print("\n===== POLICY ITERATION =====")
print("Optimal Policy:", opt_policy)
print("Optimal Value:", opt_V)
```

```
===== POLICY ITERATION =====
Optimal Policy: [0 0 0]
Optimal Value: [21.57891224 18.42102102  0.        ]
```

In [10]:

```python
#Value Iteration
def value_iteration(P, R, gamma=0.9, theta=1e-5):
    V = np.zeros(len(P))
    while True:
        delta = 0
        for s in range(len(P)):
            v = V[s]
            V[s] = max(
                R[s][a] + gamma * sum(
                    prob * V[next_state] for next_state, prob in P[s][a]
                )
                for a in P[s]
            )
```

```
            delta = max(delta, abs(v - V[s]))
        if delta < theta:
            break
    return V

V_opt = value_iteration(P, R)

print("\n===== VALUE ITERATION =====")
print("Optimal Values:", V_opt)
```

```
===== VALUE ITERATION =====
Optimal Values: [21.57890998 18.42101898  0.        ]
```

In [13]:

```
#Monte Carlo method
import numpy as np
import gymnasium as gym
env = gym.make("FrozenLake-v1")

def monte_carlo(env, episodes=500, gamma=0.9):
    returns = {}
    V = np.zeros(env.observation_space.n)

    for ep in range(episodes):
        episode = []
        state = env.reset()[0] if isinstance(env.reset(), tuple) else env.rese
        done = False

        while not done:
            action = env.action_space.sample()
            step = env.step(action)

            if len(step) == 5:  # gymnasium
                next_state, reward, done, _, _ = step
            else:  # gym
                next_state, reward, done, _ = step

            episode.append((state, reward))
            state = next_state

        # Compute returns
        G = 0
        for state, reward in reversed(episode):
            G = gamma * G + reward
            if state not in returns:
                returns[state] = []
            returns[state].append(G)
            V[state] = np.mean(returns[state])

    return V
print("\n===== MONTE CARLO OUTPUT =====")
V_mc = monte_carlo(env)

for i, v in enumerate(V_mc):
    print(f"State {i}: {v:.4f}")
```

```
print("\nGrid View:")
print(V_mc.reshape(4, 4))
```

```
===== MONTE CARLO OUTPUT =====
State 0: 0.0058
State 1: 0.0057
State 2: 0.0071
State 3: 0.0000
State 4: 0.0043
State 5: 0.0000
State 6: 0.0326
State 7: 0.0000
State 8: 0.0171
State 9: 0.0832
State 10: 0.1164
State 11: 0.0000
State 12: 0.0000
State 13: 0.1679
State 14: 0.3227
State 15: 0.0000

Grid View:
[[0.00575079 0.00571275 0.00714501 0.        ]
 [0.00431737 0.         0.03264583 0.        ]
 [0.01706469 0.08318286 0.11640146 0.        ]
 [0.         0.16791841 0.32274516 0.        ]]
```

In [14]:

```
#TD Learning
def td_learning(env, episodes=500, alpha=0.1, gamma=0.9):
    V = np.zeros(env.observation_space.n)

    for ep in range(episodes):
        state = env.reset()[0] if isinstance(env.reset(), tuple) else env.rese
        done = False

        while not done:
            action = env.action_space.sample()
            step = env.step(action)

            if len(step) == 5:
                next_state, reward, done, _, _ = step
            else:
                next_state, reward, done, _ = step

            V[state] += alpha * (reward + gamma * V[next_state] - V[state])
            state = next_state

    return V
print("\n===== TD LEARNING OUTPUT =====")
V_td = td_learning(env)

for i, v in enumerate(V_td):
    print(f"State {i}: {v:.4f}")
```

```
print("\nGrid View:")
print(V_td.reshape(4, 4))
```

```
===== TD LEARNING OUTPUT =====
State 0: 0.0016
State 1: 0.0020
State 2: 0.0044
State 3: 0.0020
State 4: 0.0036
State 5: 0.0000
State 6: 0.0187
State 7: 0.0000
State 8: 0.0121
State 9: 0.0349
State 10: 0.1672
State 11: 0.0000
State 12: 0.0000
State 13: 0.0936
State 14: 0.2821
State 15: 0.0000

Grid View:
[[0.00160763 0.00196983 0.00438906 0.00201309]
 [0.00357205 0.         0.01869542 0.        ]
 [0.01205357 0.03494876 0.16721409 0.        ]
 [0.         0.09359023 0.2820697  0.        ]]
```

In [15]:

```
# Q Learning
def q_learning(env, episodes=2000, alpha=0.1, gamma=0.9, epsilon=0.1):
    Q = np.zeros((env.observation_space.n, env.action_space.n))

    for ep in range(episodes):
        state = env.reset()[0] if isinstance(env.reset(), tuple) else env.rese
        done = False

        while not done:
            # Epsilon-greedy
            if np.random.rand() < epsilon:
                action = env.action_space.sample()
            else:
                action = np.argmax(Q[state])

            step = env.step(action)

            if len(step) == 5:
                next_state, reward, done, _, _ = step
            else:
                next_state, reward, done, _ = step

            Q[state, action] += alpha * (
                reward + gamma * np.max(Q[next_state]) - Q[state, action]
            )
```

```
            state = next_state

    return Q
print("\n===== Q-LEARNING OUTPUT =====")
Q = q_learning(env)

print("\nQ-table:")
print(Q)

# Derived policy
policy = np.argmax(Q, axis=1)

print("\nDerived Policy (State → Action):")
for i, a in enumerate(policy):
    print(f"State {i} → Action {a}")

print("\nPolicy Grid:")
print(policy.reshape(4, 4))

print("\nValue Grid:")
print(np.max(Q, axis=1).reshape(4, 4))
```

```
===== Q-LEARNING OUTPUT =====

Q-table:
[[0.0658817  0.05191282 0.04510739 0.04510349]
 [0.02366171 0.01683709 0.00801402 0.01482261]
 [0.06240506 0.         0.01708551 0.00560697]
 [0.00399729 0.         0.         0.        ]
 [0.07709291 0.05333101 0.06235539 0.05620251]
 [0.         0.         0.         0.        ]
 [0.07391131 0.02943982 0.08981254 0.0068085 ]
 [0.         0.         0.         0.        ]
 [0.07517323 0.10900145 0.10775678 0.13340079]
 [0.14560731 0.19837895 0.16196825 0.09467023]
 [0.21629726 0.18546395 0.18345618 0.05061517]
 [0.         0.         0.         0.        ]
 [0.         0.         0.         0.        ]
 [0.19009433 0.264578   0.3289031  0.22364422]
 [0.23560476 0.46159612 0.53154894 0.44726541]
 [0.         0.         0.         0.        ]]

Derived Policy (State → Action):
State 0 → Action 0
State 1 → Action 0
State 2 → Action 0
State 3 → Action 0
State 4 → Action 0
State 5 → Action 0
State 6 → Action 2
State 7 → Action 0
State 8 → Action 3
State 9 → Action 1
State 10 → Action 0
State 11 → Action 0
State 12 → Action 0
State 13 → Action 2
State 14 → Action 2
State 15 → Action 0

Policy Grid:
[[0 0 0 0]
 [0 0 2 0]
 [3 1 0 0]
 [0 2 2 0]]

Value Grid:
[[0.0658817  0.02366171 0.06240506 0.00399729]
 [0.07709291 0.         0.08981254 0.        ]
 [0.13340079 0.19837895 0.21629726 0.        ]
 [0.         0.3289031  0.53154894 0.        ]]
```

In [ ]: